GSE173955 RNA-seq Analysis

Raman Butta

2026-02-05

Contents

1. Loading the libraries . . . 1
2. Downloading and preparing the data . . . 1
3. Differential expression analysis . . . 3
4. Literature check . . . 5
5. Data visualization/QC . . . 6
6. Gene-level Heatmap . . . 7
7. Volcano Plot . . . 8
8. Over-Representation Analysis (ORA) . . . 9
9. Gene Set Enrichment Analysis (GSEA) . . . 16
10. Conclusion . . . 18

In this assignment, I will analyze the gene expression dataset GSE173955 using R. The dataset contains RNA-seq data from a cohort of 8 Alzheimer's Disease (AD) positive and 10 non-AD (control) patients. The goal is to identify differentially expressed genes between the two groups, and conduct useful downstream analyses such as functional enrichment and pathway analysis.

Experimental design

GSE173955 is a bulk RNA-sequencing dataset generated from post-mortem human brain tissue, comprising samples from individuals diagnosed with Alzheimer's disease (AD) and neurologically normal controls. The study follows an unpaired case–control design, with each sample representing an independent biological replicate from a single individual. Gene expression was quantified as raw read counts at the gene level.

1. Loading the libraries

2. Downloading and preparing the data

```
# --- LOAD DATA ---
counts <- read.table("GSE173955_raw_counts_GRCh38.p13_NCBI.tsv",
                     header = TRUE, row.names = 1, sep = "\t", check.names = FALSE)

# Load metadata from GEO
gse <- getGEO("GSE173955", GSEMatrix = TRUE)
```

```
## Found 1 file(s)
```

```
## GSE173955_series_matrix.txt.gz
```

```r
sampleInfo <- pData(gse[[1]])


# --- PREPARE METADATA ---
# Defining 'condition' column based on 'title' field
sampleInfo$condition <- factor(ifelse(grepl("non", sampleInfo$title, ignore.case = TRUE), "Control", "AD
# To ensure Control is the reference level (denominator in FoldChange)
sampleInfo$condition <- relevel(sampleInfo$condition, ref = "Control")

# Check if groups are created correctly
table(sampleInfo$condition)
```

```
##
## Control      AD
##      10       8
```

From `annotation` in `gse` we confirm that all data has been generated from the GPL18460 platform.

We can also check that `colnames(counts)` match with the rownames of sampleInfo.

There are 39,000+ gene symbols (rows) in `counts` matrix. That's because a gene may have multiple isoforms. But the gene symbols in `counts` are not in a standard format. So we will convert them to Ensembl IDs.

```r
# --- CONVERT GENE SYMBOLS TO ENSEMBL IDs ---
# Here I map Entrez IDs → Gene Symbols in counts
# This uses org.Hs.eg.db and mapIds
# It filters to keep only rows with a valid symbol (removes ~5-10% unmapped like some pseudogenes/lncRN
# Handles potential duplicates by taking the first symbol

library(org.Hs.eg.db)  # Ensure loaded

# Map Entrez (rownames) to SYMBOL
symbols <- mapIds(org.Hs.eg.db,
                  keys = rownames(counts),
                  column = "SYMBOL",
                  keytype = "ENTREZID",
                  multiVals = "first")  # Takes first if any 1:Many
```

```
## 'select()' returned 1:1 mapping between keys and columns
```

```r
# After filtering NAs, find which symbols are duplicates
keep <- !is.na(symbols)
counts_filtered <- counts[keep, ]
symbols_filtered <- symbols[keep]

# Identify unique symbols (this keeps the FIRST occurrence)
dup_mask <- !duplicated(symbols_filtered)

# Apply the final filter
```

```r
counts_final <- counts_filtered[dup_mask, ]
rownames(counts_final) <- symbols_filtered[dup_mask]

# Quick checks
print(head(rownames(counts_final)))          # Should show symbols
```

```
## [1] "DDX11L1"     "WASH7P"      "MIR6859-1"   "MIR1302-2HG" "MIR1302-2"
## [6] "FAM138A"
```

```r
print(length(unique(rownames(counts_final))))  # Should be close to original nrow(counts)
```

```
## [1] 37673
```

This prepares the count matrix with gene symbols as rownames, ready for DESeq2 analysis.

Note that we have 37673 rows (genes) which is greater than the ~20000 genes in our genome because of multiple transcripts/isoforms per gene, ncRNAs and pseudogenes.

3. Differential expression analysis

```r
# --- RUN DESEQ2 ---

dds <- DESeqDataSetFromMatrix(countData = counts_final,
                              colData = sampleInfo,
                              design = ~ condition)

dds <- DESeq(dds)
```

```
## estimating size factors

## estimating dispersions

## gene-wise dispersion estimates

## mean-dispersion relationship

## final dispersion estimates

## fitting model and testing

## -- replacing outliers and refitting for 201 genes
## -- DESeq argument 'minReplicatesForReplace' = 7
## -- original counts are preserved in counts(dds)

## estimating dispersions

## fitting model and testing
```

```r
res <- results(dds)

# --- FILTER RESULTS ---
# Filter for significant genes (Adjusted P < 0.05 & Fold Change > 1.5)
sigGenes <- res[which(res$padj < 0.05 & abs(res$log2FoldChange) > 0.58), ]

# View summary
summary(res)
```

```
##
## out of 33861 with nonzero total read count
## adjusted p-value < 0.1
## LFC > 0 (up)       : 3544, 10%
## LFC < 0 (down)     : 3718, 11%
## outliers [1]       : 0, 0%
## low counts [2]     : 8485, 25%
## (mean count < 1)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

```r
head(sigGenes)
```

```
## log2 fold change (MLE): condition AD vs Control
## Wald test p-value: condition AD vs Control
## DataFrame with 6 rows and 6 columns
##                baseMean log2FoldChange     lfcSE      stat      pvalue
##               <numeric>      <numeric> <numeric> <numeric>   <numeric>
## LINC00115      51.21672       0.807943  0.272510   2.96482 0.003028583
## LINC02593      20.73961       1.896609  0.498616   3.80375 0.000142522
## LOC107985728   10.57300       1.837740  0.645527   2.84688 0.004414943
## C1QTNF12        8.81327       1.150116  0.431919   2.66280 0.007749324
## LOC107985729    6.97551       1.200783  0.447550   2.68302 0.007296126
## TMEM240       116.86751      -0.994646  0.274703  -3.62081 0.000293679
##                     padj
##                <numeric>
## LINC00115    0.02413106
## LINC02593    0.00376747
## LOC107985728 0.03025601
## C1QTNF12     0.04294826
## LOC107985729 0.04148747
## TMEM240      0.00571526
```

```r
# --- SAVE RESULTS ---
write.csv(as.data.frame(res), file = "GSE173955_DESeq2_results.csv")
write.csv(as.data.frame(sigGenes), file = "GSE173955_Significant_Genes.csv")
# Prepare expression set for downstream analysis
exprSet <- counts(dds, normalized = TRUE)
```

`dds` is a DESeqDataSet, another S4 object (like the `gse` ExpressionSet, but RNA-seq aware).

The DESeq2 pipeline looks like : - Build `dds` DESeqDataSet object - Run `DESeq()` - Extract `res` for gene-level inference - Use `vst(dds)` for PCA & heatmaps

4. Literature check

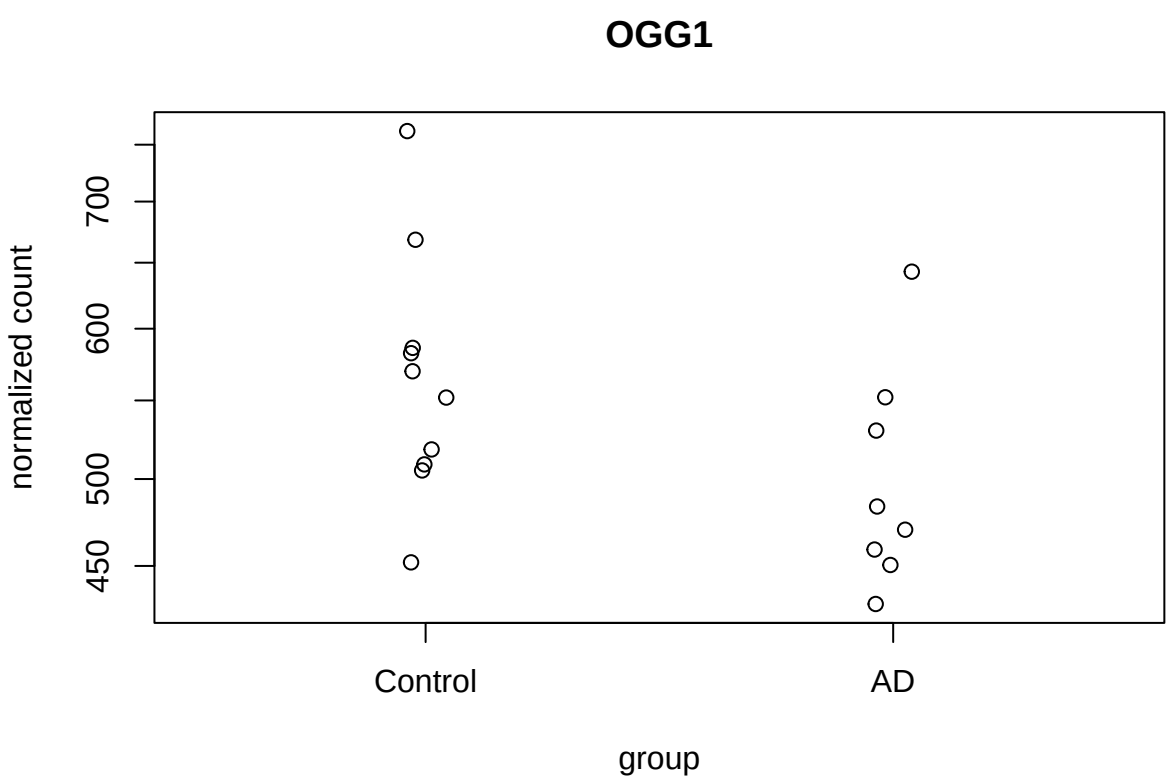
The corresponding paper of this dataset in GEO highlights that OGG1 gene minimizes the accumulation of 8-oxoG (a common type of oxidative damage), and its downregulation has been implicated in AD.

To check if the gene OGG1 is downregulated in AD samples as per literature, we can look it up in the results.

```
# Check OGG1 status
ogg1_res <- res[rownames(res) == "OGG1", ]
print(ogg1_res)
```

```
## log2 fold change (MLE): condition AD vs Control
## Wald test p-value: condition AD vs Control
## DataFrame with 1 row and 6 columns
##       baseMean log2FoldChange     lfcSE      stat    pvalue      padj
##      <numeric>      <numeric> <numeric> <numeric> <numeric> <numeric>
## OGG1   539.723      -0.181348  0.114571  -1.58284  0.113459  0.248941
```

```
# Visual check of OGG1 counts across groups
plotCounts(dds, gene="OGG1", intgroup="condition")
```



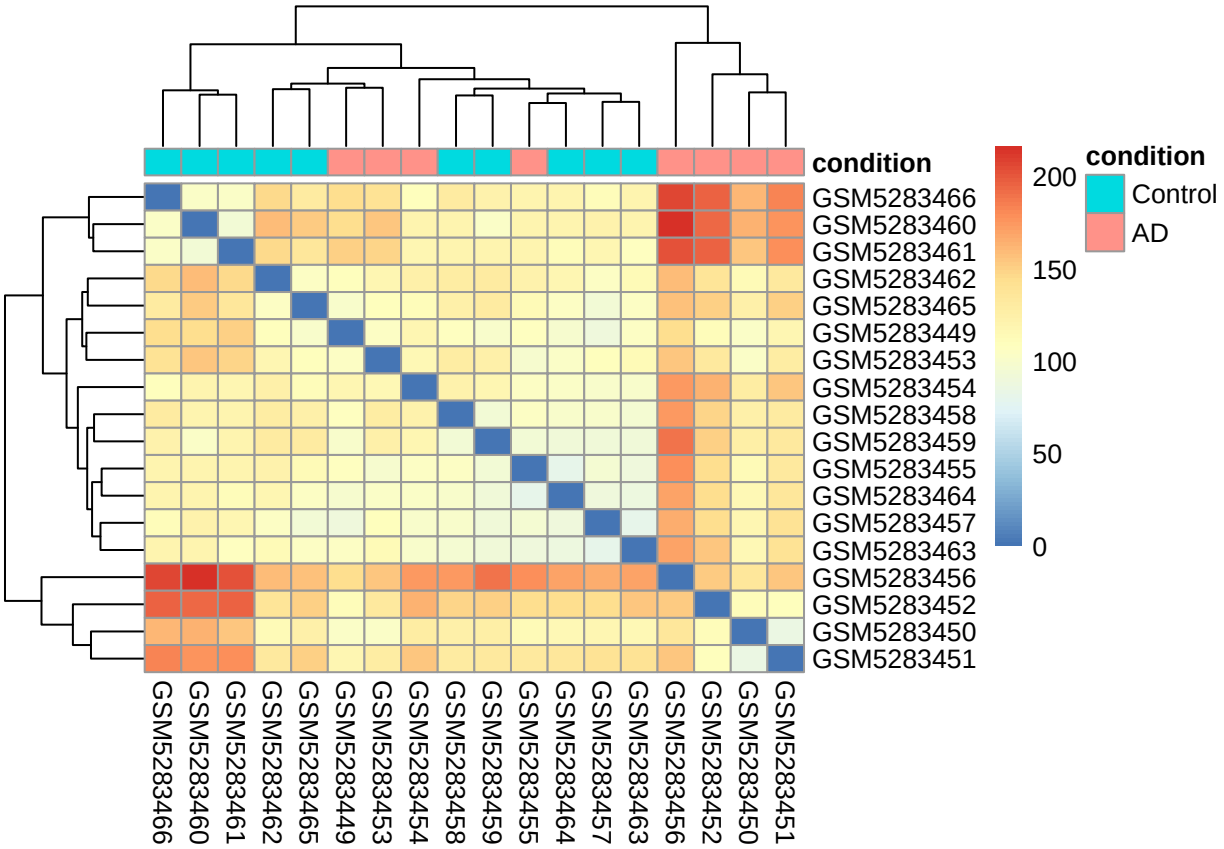
This confirms that OGG1 is indeed downregulated in AD samples compared to controls, consistent with the literature.

5. Data visualization/QC

We non-blind VST-normalize the data and plot PCA and Sample Distance Heatmap to assess sample clustering and QC.

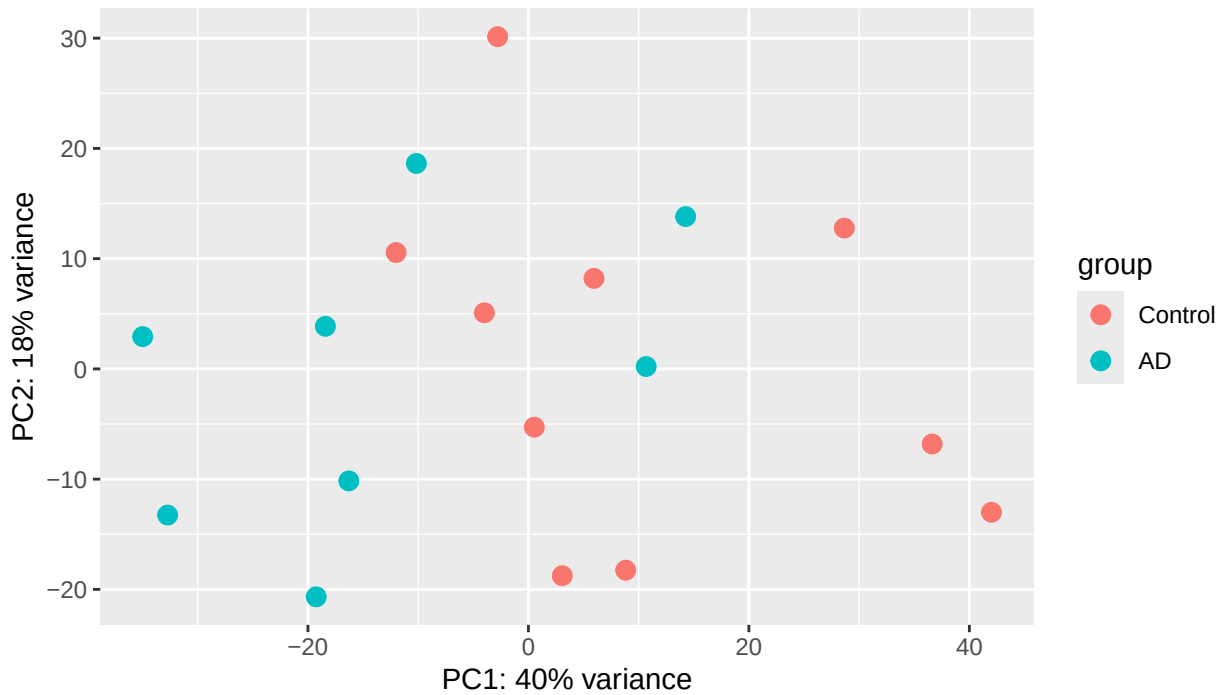
For this we'll use the `vst()` function from DESeq2 to perform variance stabilizing transformation on `dds`. `assay(vsd)` converts vsd object into a vsd table. We then transpose it because we want to find distances between samples and not genes. The `dist()` function computes the Euclidean distance between samples based on their VST-normalized expression profiles. We convert this distance object into a matrix for visualization with `pheatmap()`. The PCA plot is generated using `plotPCA()` which uses the VST-normalized data to visualize sample clustering based on the top principal components.

```
vsd <- vst(dds, blind=FALSE)
sampleDists <- dist(t(assay(vsd)))
sampleDistMatrix <- as.matrix(sampleDists)
# Sample distance heatmap
pheatmap(sampleDistMatrix,
         annotation_col = as.data.frame(colData(vsd)[, "condition", drop = FALSE]))
```




```
# PCA plot
plotPCA(vsd, intgroup = "condition")
```

```
## using ntop=500 top features by variance
```



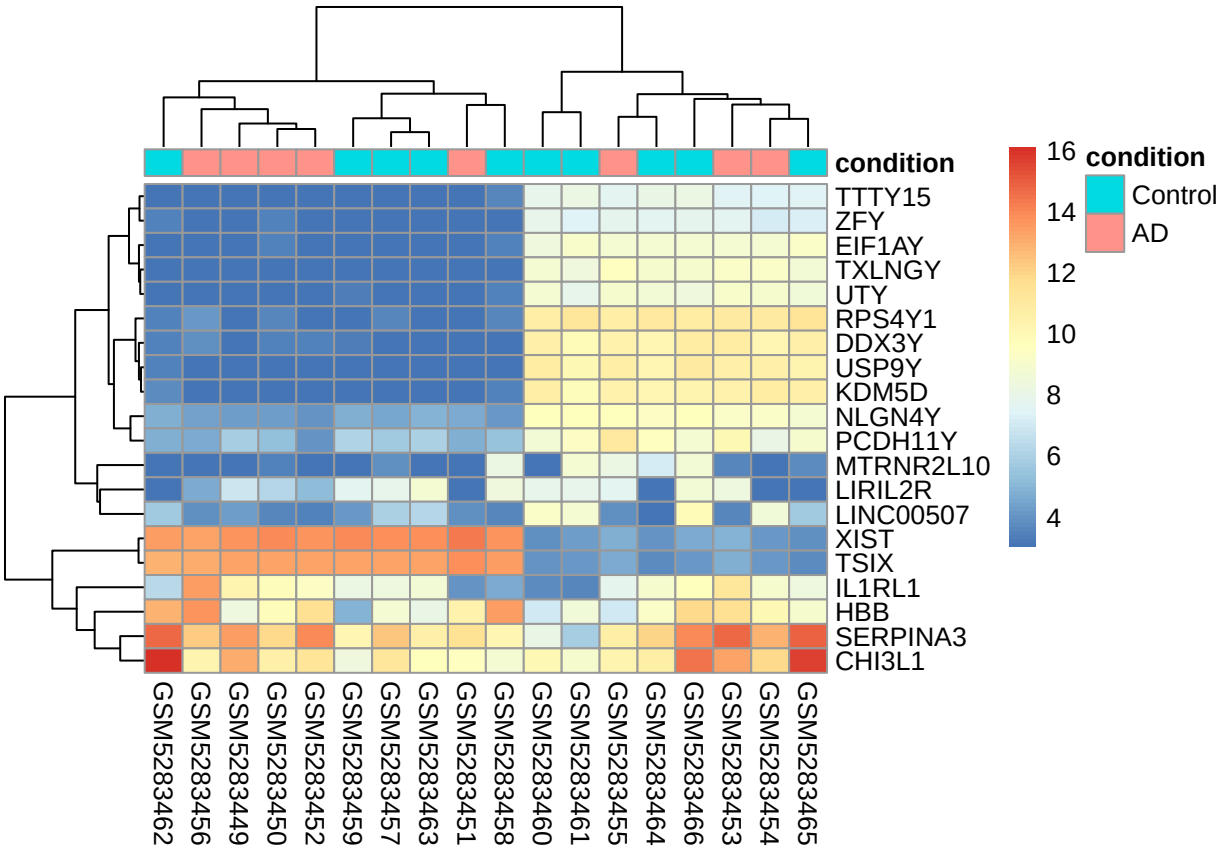
In PCA, PC1 explains ~38% of variance, PC2 ~18% which is quite good. The samples show partial separation between Control (red) and AD (blue/teal). Most Control samples cluster toward the right/positive PC1 side. Most AD samples are shifted left/negative PC1 or mixed in the middle. This is understandable as AD has heterogeneity. There are no tight clusters unrelated to condition (i.e. no batch effects). Further, there's noticeable overlap in the center, and a few outliers but no extreme outliers. Hence QC is good to proceed.

In the Sample Distance Heatmap, we see the Euclidean distance between samples based on VST-normalized expression. We see atleast some clustering of samples based on condition. And there are no single sample which is red against all other samples. This shows good QC. We further see that only a subset of AD patients drive most of the separation. This means that AD is heterogeneous, thus it doesn't rewire the transcriptome in all patients the same way.

6. Gene-level Heatmap

Next, we plot a heatmap of the top 20 most variable genes across all samples to visualize expression patterns.

```
# Identify top 20 most variable genes
topVarGenes <- head(order(rowVars(assay(vsd)), decreasing = TRUE),
                    20)
# Plot heatmap
pheatmap(assay(vsd)[topVarGenes, ],
         cluster_rows = TRUE,
         cluster_cols = TRUE,
         annotation_col = as.data.frame(colData(vsd)[, "condition", drop = FALSE
         ]))
```



This heatmap shows the gene expression (not sample distance) of the top 20 DEGs across all samples. We can see here that many genes are upregulated in AD (red) compared to controls (blue), while a few are downregulated. The samples cluster reasonably well by condition, with some heterogeneity as expected.

7. Volcano Plot

Next, we create a volcano plot using ggplo2, to visualize the overall differential expression results.

Since ggplot2 works on plain dataframes, we first convert the DESeq2 results object `res` into a dataframe. We then create a new column `significant` to indicate whether each gene is significantly differentially expressed based on our criteria (adjusted p-value < 0.05 and absolute log2 fold change > 0.58). The 0.58 value filters only those genes which change by atleast 1.5x, up or down. Finally, we use ggplot2 to create the volcano plot, coloring significant genes in red/blue and non-significant genes in grey.

```
# Volcano plot
res_df <- as.data.frame(res)
res_df$significant <- dplyr::case_when(
  res_df$padj < 0.05 & res_df$log2FoldChange >  0.58 ~ "Upregulated",
  res_df$padj < 0.05 & res_df$log2FoldChange < -0.58 ~ "Downregulated",
  TRUE                                                ~ "Not significant"
)

ggplot(res_df, aes(x = log2FoldChange,
                   y = -log10(padj),
                   color = significant)) +
  geom_point(alpha = 0.5) +
```

```r
  scale_color_manual(values = c(
    "Upregulated"     = "red",
    "Downregulated"   = "blue",
    "Not significant" = "grey"
  )) +
  theme_minimal() +
  labs(
    title = "Volcano Plot of Differential Expression",
    x = "Log2 Fold Change (AD vs Control)",
    y = "-Log10 Adjusted P-value"
  ) +
  theme(legend.position = "top")
```

```
## Warning: Removed 12296 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

The volcano plot shows the distribution of all genes based on their log2 fold change and adjusted p-value. Genes that are significantly differentially expressed (red & blue points) are clearly separated from non-significant genes (grey points). We can see a good number of genes with high fold changes and low p-values, indicating robust differential expression between AD and control samples.

8. Over-Representation Analysis (ORA)

We will perform Gene Ontology (GO) enrichment analysis on the **set** of significant DEGs (irrespective of their counts) to identify overrepresented biological processes (BPs). Hence while DESeq2 used the data, GO

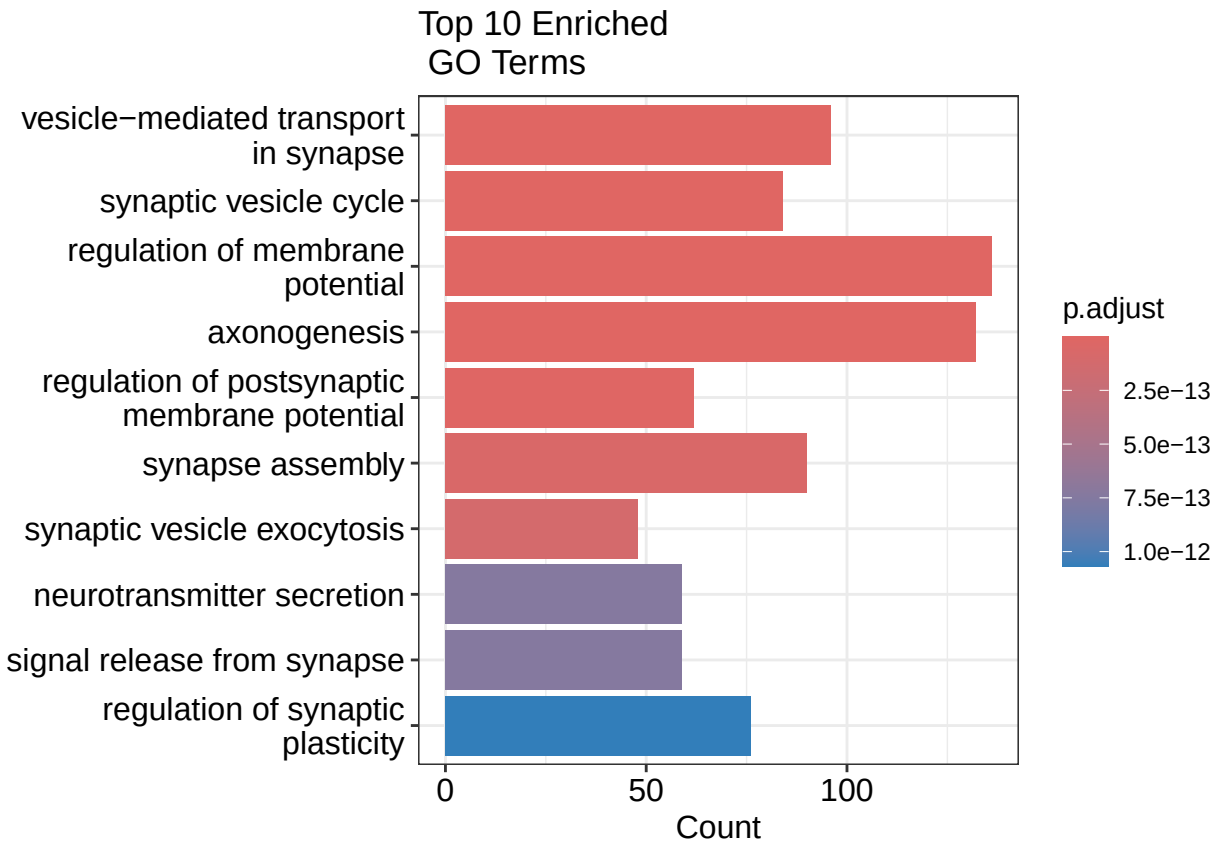
enrichment uses only the gene list.

```r
# Prepare gene list for enrichment
sigGeneSymbols <- rownames(sigGenes)
# Map gene symbols back to Entrez IDs for GO to work
entrezIDs <- mapIds(org.Hs.eg.db,
                    keys = sigGeneSymbols,
                    column = "ENTREZID",
                    keytype = "SYMBOL",
                    multiVals = "first")
```

```
## 'select()' returned 1:1 mapping between keys and columns
```

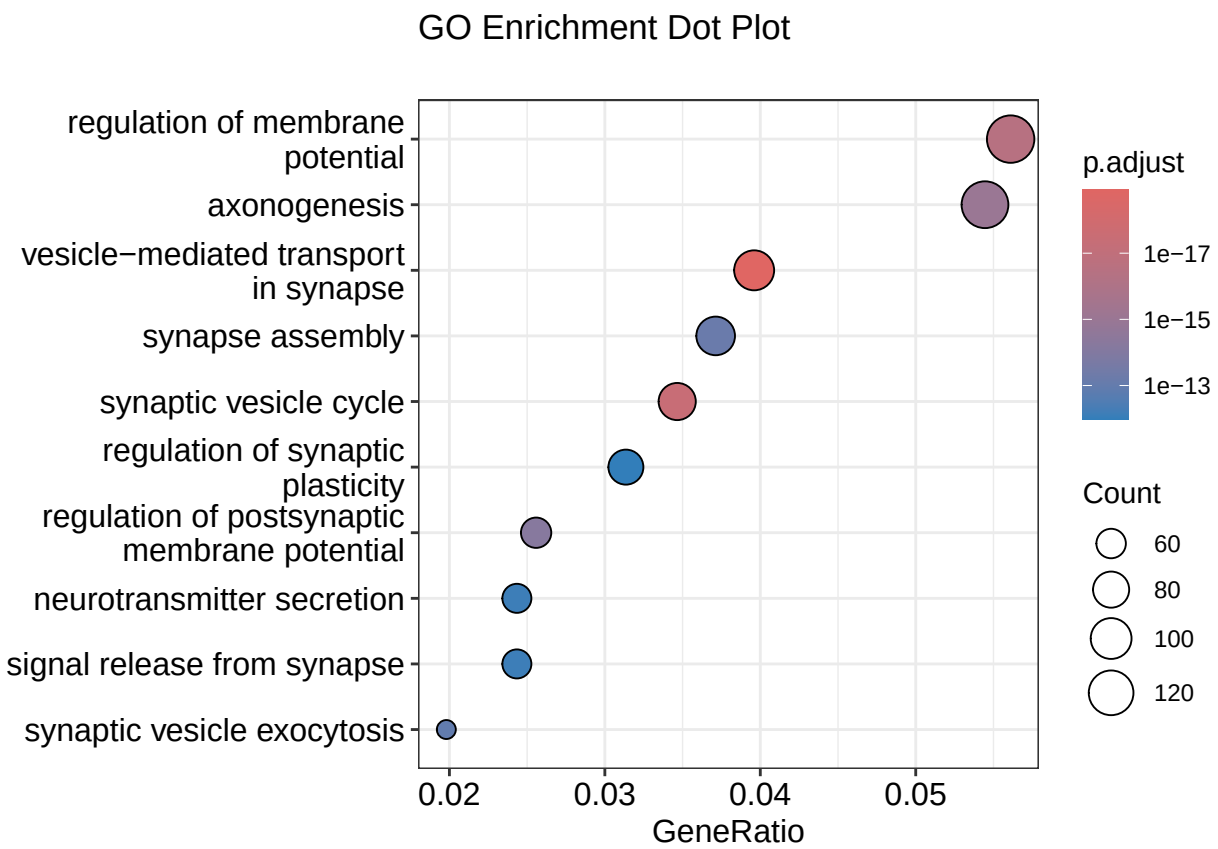
```r
entrezIDs <- na.omit(entrezIDs)
# GO enrichment analysis
ego <- enrichGO(gene = entrezIDs,
                OrgDb = org.Hs.eg.db,
                keyType = "ENTREZID",
                ont = "BP",
                pAdjustMethod = "BH",
                pvalueCutoff = 0.05,
                qvalueCutoff = 0.05)
# Plot top 10 enriched GO terms
barplot(ego, showCategory = 10, title = "Top 10 Enriched
 GO Terms")
```

```
## Warning in (function (model, data, ...) : Arguments in `...` must be used.
## x Problematic argument:
## * by = "Count"
## i Did you misspell an argument name?
```



The above plot shows that the top enriched GO terms among the DEGs include processes related to membrane potential, axonogenesis and cell signaling. This suggests that these biological processes may be significantly altered in AD patients compared to controls. Further, we can visualize the GO enrichment results using a dot plot.

```
# Dot plot of GO enrichment
dotplot(ego, showCategory = 10, title = "GO Enrichment Dot Plot
")
```



The dot plot provides a visual representation of the enriched GO terms, showing the gene ratio and significance for each term. The size of the dots represents the number of genes associated with each GO term, while the color indicates the adjusted p-value. This visualization helps to quickly identify the most relevant biological processes affected in AD.

We can use KEGG pathway analysis to further explore the pathways enriched among the DEGs.

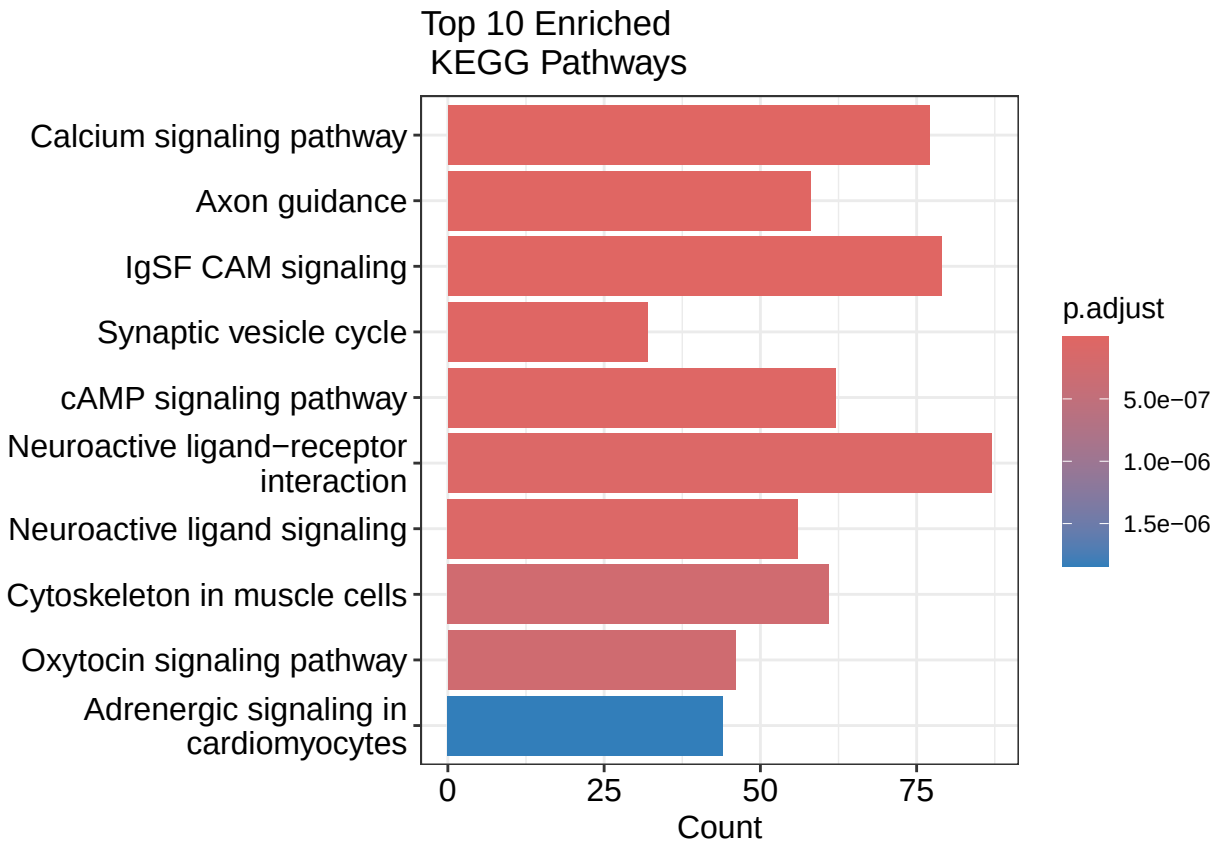
```
# KEGG pathway enrichment analysis
kegg <- enrichKEGG(gene = entrezIDs,
                   organism = 'hsa',
                   pvalueCutoff = 0.05)
```

```
## Reading KEGG annotation online: "https://rest.kegg.jp/link/hsa/pathway"...
```

```
## Reading KEGG annotation online: "https://rest.kegg.jp/list/pathway/hsa"...
```

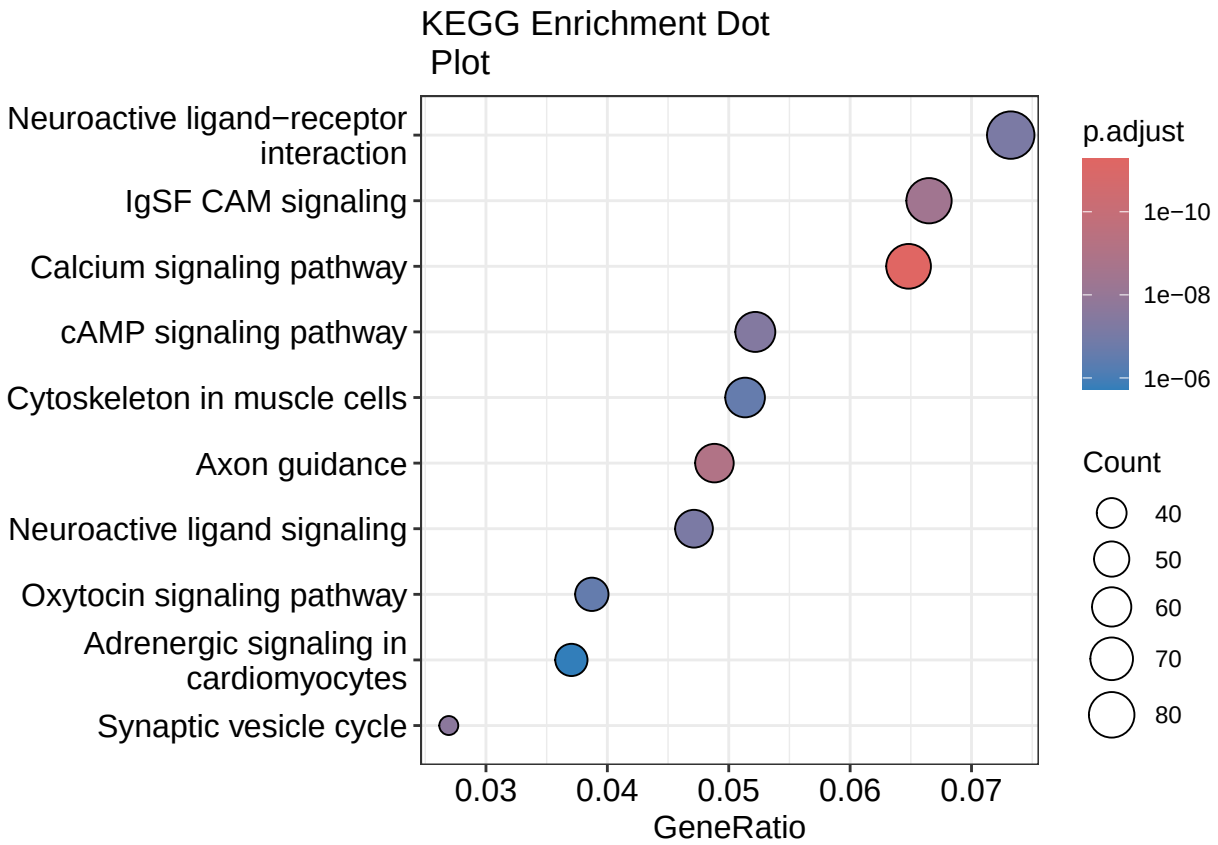
```
# Plot top 10 enriched KEGG pathways
barplot(kegg, showCategory = 10, title = "Top 10 Enriched
 KEGG Pathways")
```

```
## Warning in (function (model, data, ...) : Arguments in `...` must be used.
## x Problematic argument:
## * by = "Count"
## i Did you misspell an argument name?
```



The KEGG pathway analysis reveals several pathways that are significantly enriched among the DEGs. This indicates that these pathways may play a crucial role in the pathogenesis of Alzheimer's disease.

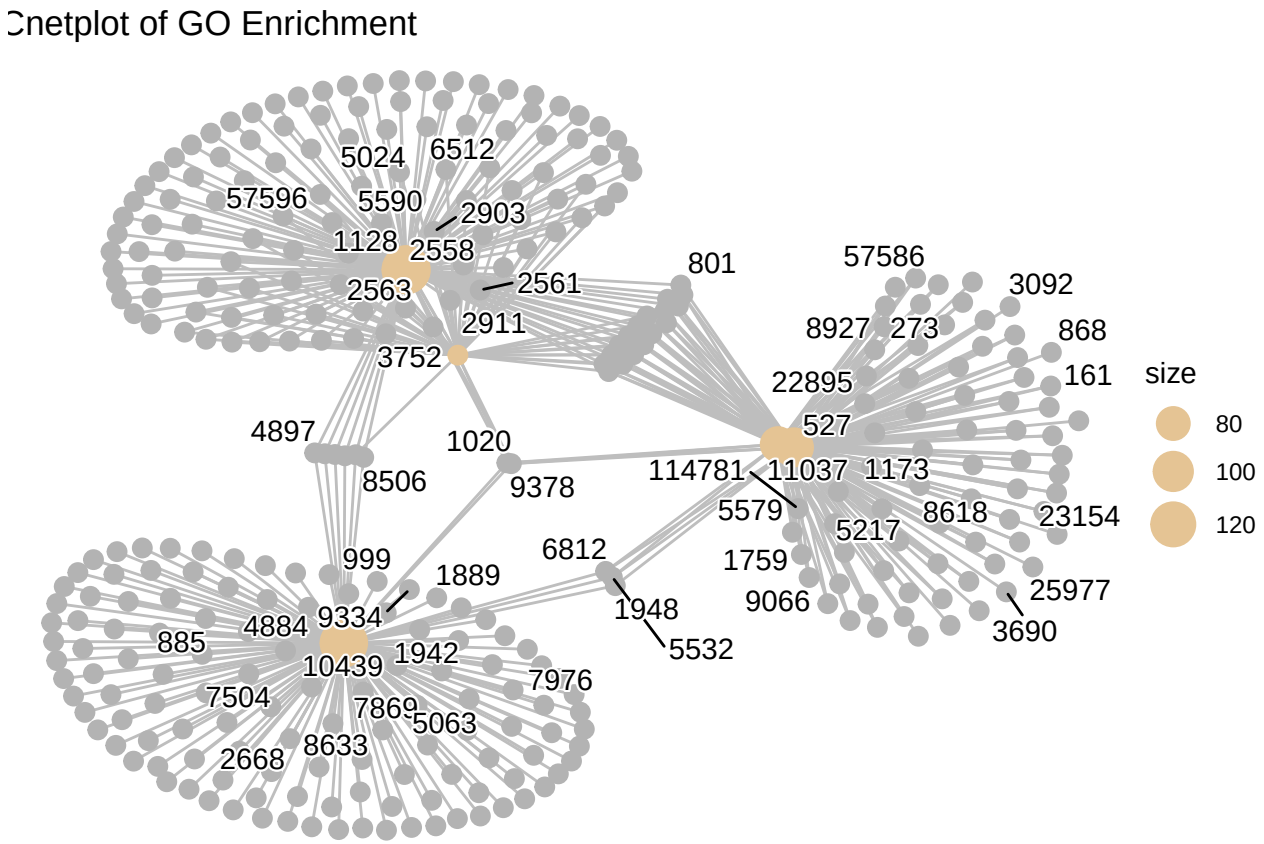
```
# Dot plot of KEGG enrichment
dotplot(kegg, showCategory = 10, title = "KEGG Enrichment Dot
 Plot")
```



The above plots show us the no. of genes involved in different BPs, but don't reflect how many genes in the set impact multiple BPs. For that we can use clusterProfiler's **cnetplot** to visualize the relationships between genes and enriched GO terms.

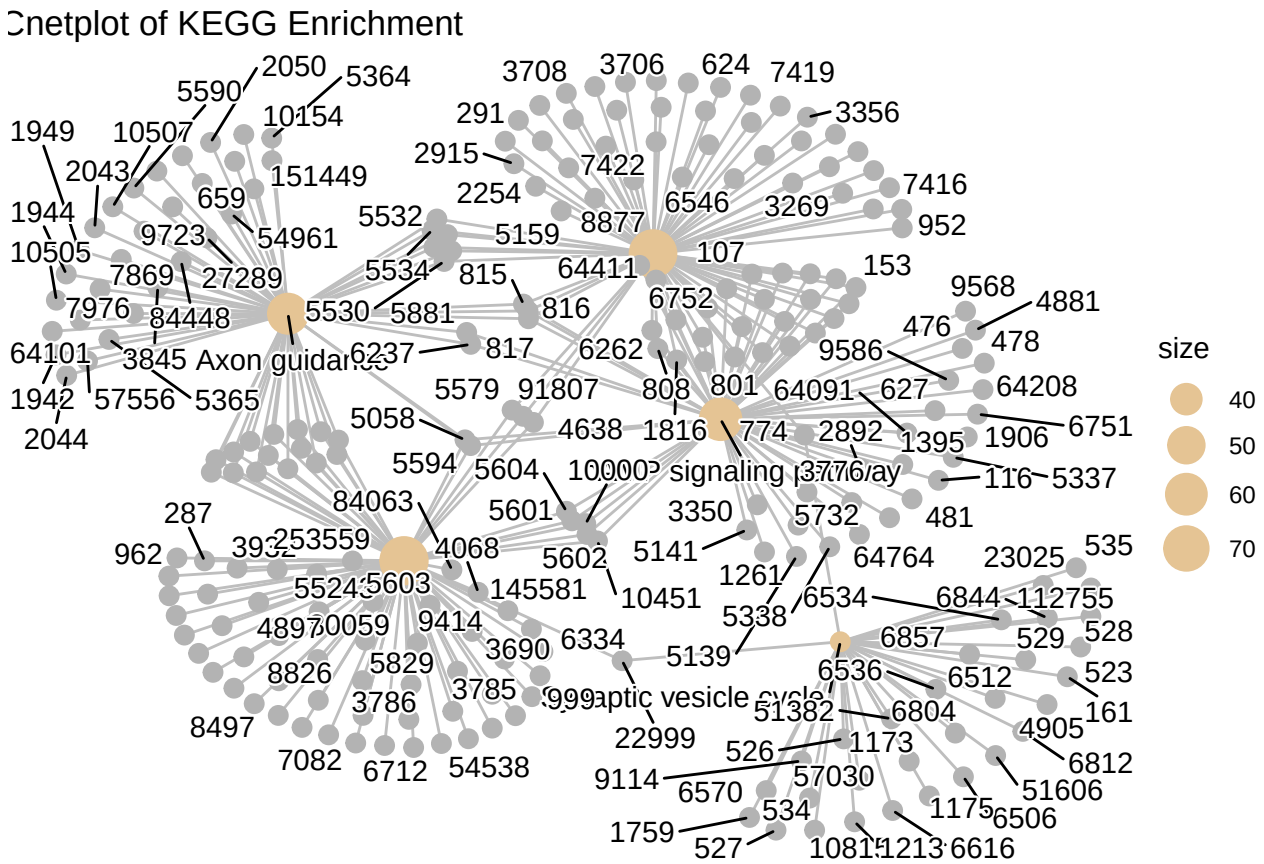
```
# Cnetplot for GO enrichment
cnetplot(ego, showCategory = 5) +
  ggtitle("Cnetplot of GO Enrichment")
```

```
## Warning: ggrepel: 288 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps
```




```
# Cnetplot for KEGG enrichment
cnetplot(kegg, showCategory = 5) +
  ggtitle("Cnetplot of KEGG Enrichment")
```

```
## Warning: ggrepel: 102 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps
```



The cnetplots illustrate the connections between the significant DEGs and the top enriched GO terms and KEGG pathways. This visualization helps to identify key genes that may be driving the enrichment of specific biological processes and pathways in Alzheimer's disease.

From the above plots, it is evident that DEGs in AD samples are enriched in interconnected synaptic and neuronal signaling processes, including synaptic vesicle cycling, axon guidance, and CAM signaling. This suggests that dysregulation of synaptic function and neuronal communication may be central to AD pathogenesis.

9. Gene Set Enrichment Analysis (GSEA)

Next, we can perform Gene Set Enrichment Analysis (GSEA) to identify pathways that are enriched in the ranked list of all genes based on their differential expression. So we need to prepare a "ranked list" of all genes based on log2 fold change, which denotes the magnitude and direction of differential expression.

Unlike ORA (Over-Representation Analysis), which only looks at a list of "significant" genes, GSEA considers all genes in your experiment.

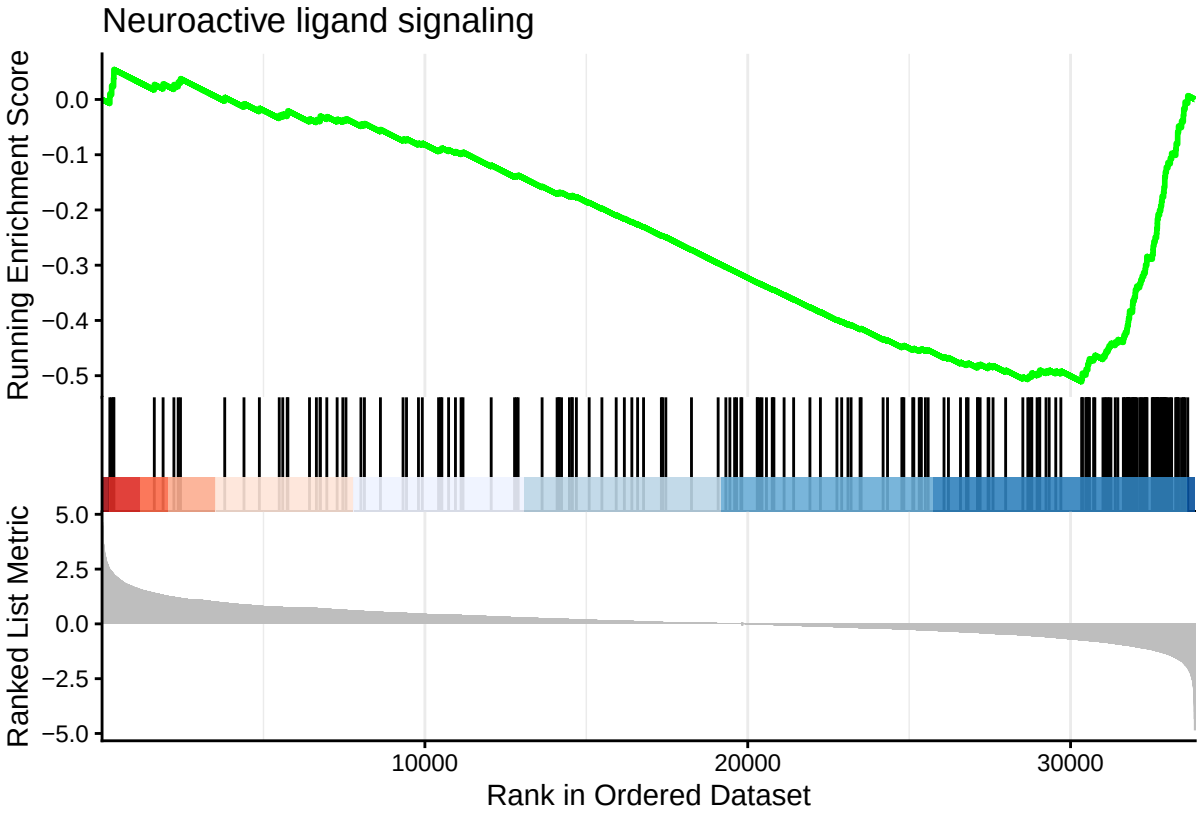
```
# 1. Map the IDs for all genes in your results
entrez_map <- mapIds(org.Hs.eg.db,
                     keys = rownames(res),
                     column = "ENTREZID",
                     keytype = "SYMBOL",
                     multiVals = "first")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```r
# 2. Create a new dataframe
valid_genes <- data.frame(
  log2FC = res$log2FoldChange,
  entrez = entrez_map
)

# 3. Remove rows where Entrez ID is NA
valid_genes <- na.omit(valid_genes)

# 4. Create the named vector for GSEA
geneList <- valid_genes$log2FC
names(geneList) <- valid_genes$entrez

# 5. Final Sort (Crucial for GSEA)
geneList <- sort(geneList, decreasing = TRUE)

# GSEA for KEGG pathways
gsea_kegg <- gseKEGG(geneList = geneList,
                     organism = 'hsa',
                     pvalueCutoff = 0.05)
```

```
## using 'fgsea' for GSEA analysis, please cite Korotkevich et al (2019).

## preparing geneSet collections...

## GSEA analysis...

## Warning in preparePathwaysAndStats(pathways, stats, minSize, maxSize, gseaParam, : There are ties in
## The order of those tied genes will be arbitrary, which may produce unexpected results.

## Warning in fgseaMultilevel(pathways = pathways, stats = stats, minSize =
## minSize, : For some of the pathways the P-values were likely overestimated. For
## such pathways log2err is set to NA.

## Warning in fgseaMultilevel(pathways = pathways, stats = stats, minSize =
## minSize, : For some pathways, in reality P-values are less than 1e-10. You can
## set the `eps` argument to zero for better estimation.

## leading edge analysis...

## done...
```

```r
# Plot GSEA results
gseaplot2(gsea_kegg, geneSetID = gsea_kegg$
ID[1], title = gsea_kegg$Description[1])
```



I created `geneList` as a named vector of log2FC. So after sorting, the top (left) of this list contains the genes upregulated in AD, and the bottom (right) of the list contains the downregulated genes. `gsea_kegg` is a processed version of `geneList`. It contains the results of GSEA, including the enrichment scores and p-values for each KEGG pathway. We can visualize the most significantly enriched pathway using `gseaplot2()`, which shows the running enrichment score (ES) across the ranked gene list and the positions of genes from that pathway.

The most significantly enriched KEGG pathway in our dataset comes out to be the Neuroactive ligand signaling pathway. Its ES curve reaches its negative peak around -0.5 towards the right side of the ranked list, indicating that genes in this pathway are predominantly downregulated in AD samples compared to controls. The vertical bars indicate the positions of genes from this pathway in the ranked list, with many of them clustered towards the bottom (downregulated) end.

These pathways may be involved in the underlying mechanisms of Alzheimer's disease and could be potential targets for therapeutic intervention.

10. Conclusion

In this analysis of the GSE173955 dataset, we identified differentially expressed genes between Alzheimer's disease (AD) patients and non-AD controls using DESeq2. We found a substantial number of genes that were significantly upregulated or downregulated in AD samples.

Overall, the differential expression and functional enrichment analyses provide insights into the molecular changes associated with Alzheimer's disease, highlighting key genes and pathways that may contribute to disease pathology. Further validation and experimental studies would be needed to confirm these findings and explore their potential as therapeutic targets.